

Web Application Development

Lab 9 – Practice: SPRING SECURITY & JWT AUTHENTICATION

Name: Đặng Thanh Lâm – ITITDK23039

Testing

*admin login

The screenshot displays a REST client interface with the following details:

- Request:**
 - Method: POST
 - URL: `http://localhost:8080/api/auth/login`
 - Body (JSON):

```
{  "username": "admin",  "password": "password123"}
```
- Response:**
 - Status: 200 OK
 - Time: 190
 - Body (JSON):

```
{  "token": "eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJhZG1pb2IiIm1hdCI6MTc2NTYxNDEwNywiZmxwIjoxNzY1NzAwNTA3fQ.mhfBnc73kG0Uji89w9UK4V0S9Q2XVHvNsM6rfGi04a0jKcduOHV-g__aJYdLPqndKTjKORsQ08RB5JI0tMXika",  "username": "admin",  "email": "admin@example.com",  "role": "ADMIN",  "type": "Bearer"}
```

At the bottom left, there is a "Select Data:" section with a text input field containing "Type to filter..."

*register new user

The screenshot displays a REST client interface with a dark theme. The top bar shows a POST request to `http://localhost:8080/api/auth/register` with a 'Send' button. Below the bar, tabs for 'Auth', 'Headers', 'Body', 'Pre-request', and 'Post-request' are visible, with 'Body' selected. The 'Body' tab shows a JSON payload with fields for username, email, password, and full name. On the right, the 'Response' tab is active, showing a 201 status and a JSON response containing user details like id, username, email, role, and creation timestamp. A 'Select Data' filter is at the bottom left.

POST `http://localhost:8080/api/auth/register` Send

Auth Headers **Body** Pre-request Post-request

☒ JSON ☐ XML ☐ x-www-form-urlencoded ☐ Text ☐ multipart/form-data

```
1 {
2   "username": "testuser1",
3   "email": "test1@example.com",
4   "password": "password123",
5   "fullName": "Test User"
6 }
```

Select Data:
Type to filter...

Status: 201 Created Time: 325 Env

Response Headers Results

```
1 {
2   "id": 5,
3   "username": "testuser1",
4   "email": "test1@example.com",
5   "fullName": "Test User",
6   "role": "USER",
7   "isActive": true,
8   "createdAt": "2025-12-13T15:24:02.8069957"
9 }
```

*protected endpoint (without token)

The screenshot displays a REST client interface with a dark theme. The top bar shows a GET request to `http://localhost:8080/api/customers` with a 'Send' button. Below this, the 'Body' tab is selected, showing an empty JSON object `{}` with line numbers 1, 2, and 3. To the right of the body, there are radio buttons for content types: `JSON` (selected), `XML`, `x-www-form-urlencoded`, `Text`, and `multipart/form-data`. At the bottom left, there is a 'Select Data:' section with a search input labeled 'Type to filter...'. On the right side, the 'Response' tab is active, showing a 401 Unauthorized status with a time of 14. The response body is a JSON object: `{ "path": "/api/customers", "error": "Unauthorized", "message": "Authentication required. Please provide valid JWT token.", "timestamp": "2025-12-13T15:26:17.713526300", "status": 401 }`. The interface also includes tabs for 'Auth', 'Headers', 'Pre-request', and 'Post-request' on the left, and 'Headers' and 'Results' on the right.

GET `http://localhost:8080/api/customers` Send

Auth Headers **Body** Pre-request Post-request

☒ JSON ☐ XML ☐ x-www-form-urlencoded ☐ Text ☐ multipart/form-data

```
1 {
2
3 }
```

Select Data:
Type to filter...

Status: 401 Unauthorized Time: 14 Env

Response Headers Results

```
1 {
2   "path": "/api/customers",
3   "error": "Unauthorized",
4   "message": "Authentication required.
5   Please provide valid JWT token.",
6   "timestamp": "2025-12-13T15:26:17.
7   713526300",
8   "status": 401
9 }
```

*protected endpoint with token

The screenshot shows a REST client interface with a GET request to `http://localhost:8080/api/users` and its response. The request headers include `Content-Type: application/json` and a Bearer token. The response is a JSON array of three user objects.

Request:

```
GET http://localhost:8080/api/users
```

Headers:

```
Content-Type: application/json
Authorization: Bearer eyJhbGciOiJIUzUxMiJ9.eyJzdWIiOiJhZG1pb2IiLCJhdCI6MTc2NTYxNDc3MCwiZmxhZS51cm50smrh1L2aJucXNAg6m549FsY3G9kKcgYVALpZyRQmupSHT0kiAg
```

Status: 200 OK Time: 1425

Response:

```
[
  {
    "id": 1,
    "username": "admin",
    "email": "admin@example.com",
    "fullName": "Admin User",
    "role": "ADMIN",
    "isActive": true,
    "createdAt": "2025-12-13T21:19:54"
  },
  {
    "id": 2,
    "username": "john",
    "email": "john@example.com",
    "fullName": "John Doe",
    "role": "USER",
    "isActive": true,
    "createdAt": "2025-12-13T21:19:54"
  },
  {
    "id": 3,
    "username": "jane",
    "email": "jane@example.com",
    "fullName": "Jane Smith",
    "role": "USER",
    "isActive": true,
    "createdAt": "2025-12-13T21:19:54"
  }
]
```

*protected endpoint without token

The screenshot shows a REST client interface with a GET request to `http://localhost:8080/api/users`. The response is a 401 Unauthorized status with a JSON body indicating authentication is required.

Request:

- Method: GET
- URL: `http://localhost:8080/api/users`
- Headers:
 - Content-Type: `application/json`

Response:

Status: 401 Unauthorized Time: 33

Response Body (JSON):

```
{
  "path": "/api/users",
  "error": "Unauthorized",
  "message": "Authentication required. Please provide valid JWT token.",
  "timestamp": "2025-12-13T15:34:38.276508600",
  "status": 401
}
```

Select Data:

Type to filter...